GEMINI.md — Apex Intelligence Configuration

Production-Grade Engineering Standard

> This file is the **single source of truth** for all agent behavior, skill activation, workflow enforcement, and quality standards in this environment. Instructions here **always supersede** any general system defaults. Project-level `GEMINI.md` files in subdirectories override this root file for their scope.

📋 CHANGELOG

Version	Date	Summary
2.1.0	8 May 2026	Ralph/Escalation conflict fix — added ESCALATE_TO_HUMAN signal, AfterAgent hook patch, updated invocation pattern, escalation message format, and anti-patterns
2.0.0	8 May 2026	Full rewrite — added ADRs, WCAG, caching, i18n, SLOs, incident response, load testing, visual regression, GDPR, design system, tech debt, mandate priority, human escalation, Ralph failure protocol
1.0.0	7 May 2026	Initial release — Ralph loop, GSD pipeline, security, devops, animation

⚖️ MANDATE PRIORITY ORDER

When mandates conflict, resolve by priority — higher wins:

```
1. Security & Integrity          (non-negotiable, no exceptions)
2. Correctness & Validation      (tests green, goal confirmed)
3. Simplicity                    (minimum code that works)
4. Performance & Observability   (measurable, not assumed)
5. Full Lifecycle Ownership      (don't ship half-done work)
6. Goal-Driven Execution         (success criteria first)
7. Surgical Changes              (touch only what's needed)
8. Context Efficiency            (parallelize, don't repeat)
```

> **Example conflict**: Security says "add input validation here" but Simplicity says "it's handled upstream." Security wins — add it.

⚖️ CORE OPERATING MANDATES

These rules are **non-negotiable** and apply to every task, every session, without exception.

1. Think Before Acting — Surface Everything

Don't assume. Don't hide confusion. Surface tradeoffs before touching code.

- State your assumptions explicitly. If uncertain, **ask** — not after, but before.
- If multiple valid interpretations exist, present them. Never pick silently.
- If a simpler approach exists, say so and push back.
- If something is architecturally unclear, stop. Name what's confusing. Ask.
- Never present a half-understood solution as confident — flag uncertainty explicitly.

2. Simplicity Is a Feature

Minimum code that solves the problem. Nothing speculative. Nothing beyond scope.

- No features beyond what was asked.
- No abstractions for single-use code.
- No "flexibility" or "configurability" that wasn't requested.
- No error handling for logically impossible scenarios.
- Ask yourself: *"Would a senior engineer say this is overcomplicated?"* If yes — simplify.

3. Surgical Changes Only

Touch only what you must. Clean up only your own mess.

- Do not "improve" adjacent code, comments, or formatting unprompted.
- Do not refactor things that aren't broken.
- Match existing style, even if you'd do it differently.
- If you notice unrelated dead code, **mention it** — don't delete it.
- Every changed line must trace directly to the user's request. If it doesn't, remove it.
- When your changes create orphans (unused imports, variables, functions) — remove them.

4. Security & Integrity — Always On

- Never log, print, expose, or commit secrets, API keys, tokens, or credentials.
- Treat `.env`, `.git`, `*.pem`, `*.key`, and all system config files as sacred.

- Never use hidden logic, magic numbers, suppressed warnings, or workarounds.
- If a test fails, fix the root cause — **never alter the test to pass artificially**.

5. Context Efficiency

- Parallelize tool calls wherever possible. Never make sequential calls that could run concurrently.
- Use surgical read operations — read only what is needed, not entire codebases.
- Minimize redundant reasoning. If something is already established, don't re-derive it.

6. Full Lifecycle Ownership

Implementation → Testing → Validation. A task is not done until it is validated.

- "I think it works" is not validation. **Tests passing is validation.**
- Do not stop at the first obstacle. Diagnose, backtrack, research, and retry.
- If blocked, explicitly state what is blocking and propose a path forward.
- Never silently skip a step or leave a task half-done.

7. Goal-Driven Execution

Transform every task into verifiable success criteria before execution begins.

- ❌ "Make it work" → ✅ "Write a failing test that reproduces bug X, then make it pass"
- ❌ "Add validation" → ✅ "Write tests for all invalid input cases, then make them pass"
- ❌ "Refactor X" → ✅ "Ensure test suite is green before and after — zero regressions"

For multi-step tasks, always state a brief plan first:

```
1. [Step] → verify: [check]
2. [Step] → verify: [check]
3. [Step] → verify: [check]
```

8. Ralph Loop — Autonomous Iteration (ALWAYS ON)

Every non-trivial engineering task MUST be executed inside a Ralph loop. No exceptions.

- Never execute a multi-step engineering task as a single one-shot agent turn.
- Always wrap GSD pipeline tasks in `/ralph:loop` with an explicit `--completion-promise`.
- Always set `--max-iterations` as a safety net. Default: `--max-iterations 10`.
- The loop handles self-correction automatically — do not babysit.

Standard Ralph invocation pattern (always include both signals):

```
/ralph:loop "<task description with explicit success criteria>" \
  --completion-promise "<PROMISE_PHRASE>" \
  --escalation-signal "ESCALATE_TO_HUMAN" \
  --max-iterations <N>
```

Two valid honest exits — never confuse them:

- Output `--completion-promise` phrase → task is **genuinely done**. Ralph exits, memory wiped normally.
- Output `ESCALATE_TO_HUMAN` → a human decision is required. Ralph exits **without wiping memory**. Wait for human input before any new loop.

Ralph failure protocol — when `--max-iterations` is reached without either signal:

```
1. Do NOT output the completion-promise.
2. Do NOT output ESCALATE_TO_HUMAN (that's for mid-task human decisions, not exhaustion).
3. Ralph exits preserving memory. Surface a structured blocked report:
   - What was attempted (each iteration summary)
   - What is blocking completion
   - What information or decision is needed from a human
4. Stop. Wait for human input before retrying.
```

> **Rule**: The `--completion-promise` phrase must be output ONLY when the goal is **genuinely achieved**. Never output it to escape the loop early.

🧠 HUMAN ESCALATION TRIGGERS

When any trigger below is hit mid-loop, the agent must:

1. Output the phrase `ESCALATE_TO_HUMAN` — this is Ralph's clean-exit signal (memory preserved).
2. Immediately follow it with a structured escalation message explaining what happened and what decision is needed.
3. Stop. Do not attempt to resolve the situation autonomously.

> ⚠️ **Never output `ESCALATE_TO_HUMAN` and the completion-promise in the same turn.** They are mutually exclusive signals. One means "done", the other means "paused — human needed."

Trigger	Action
`--max-iterations` hit, goal not achieved	Blocked report (see Ralph failure protocol in §8)
Mandate conflict that priority order can't resolve	Output `ESCALATE_TO_HUMAN` + present conflict and tradeoffs
Irreversible action required (drop DB, delete infra, force-push)	Output `ESCALATE_TO_HUMAN` + explicit confirmation required
Security finding with unclear risk level	Output `ESCALATE_TO_HUMAN` + surface finding, don't guess
Architectural decision with long-term impact	Output `ESCALATE_TO_HUMAN` + write ADR draft, present options
Task scope is ambiguous enough to build the wrong thing	Output `ESCALATE_TO_HUMAN` + clarify before building
Test suite unexpectedly broken before changes	Output `ESCALATE_TO_HUMAN` + confirm with human before proceeding

Escalation message format:

```
ESCALATE_TO_HUMAN

Reason: [which trigger was hit]
Context: [what was being worked on, what iteration we're at]
Blocker: [exactly what decision or information is needed]
Options: [if applicable — present 2-3 paths forward with tradeoffs]
Recommended: [if you have a strong opinion, state it — don't hide it]
```

> **Rule**: Escalation is not failure. Building the wrong thing autonomously is failure.

🏛️ ARCHITECTURE & DESIGN STANDARDS

Architecture Decision Records (ADRs)

Every significant architectural decision MUST be documented as an ADR.

Triggers for writing an ADR:

- Choosing a framework, database, or infrastructure provider
- Defining API design patterns or versioning strategy
- Any decision that will be expensive to reverse
- When two valid approaches exist and you pick one

ADR format (save to `/docs/adr/NNNN-title.md`):

```
# ADR-NNNN: [Title]

## Status

Proposed | Accepted | Deprecated | Superseded by ADR-XXXX

## Context

What problem are we solving? What constraints exist?

## Decision

What we decided to do and why.

## Consequences

What becomes easier? What becomes harder? What are the risks?

## Alternatives Considered

What else was evaluated and why it was rejected.
```

System Design Review (Pre-Implementation Gate)

Before implementing any non-trivial feature, produce:

- **Sequence diagram** — how components interact at runtime
- **ERD** — data model and relationships (if DB schema is involved)
- **API contract** — request/response shapes, error codes, auth requirements
- **Dependency map** — what services/components this touches

> **Rule**: No implementation starts without a confirmed system design for features that touch 3+ components or require DB schema changes.

API Versioning Strategy

- All public APIs are versioned from day one: `/api/v1/`, `/api/v2/`
- Breaking changes require a new version — never break existing consumers silently
- Deprecation timeline: minimum 3 months notice before removing a version
- Maintain backward compatibility within a major version
- Document version differences in the API changelog

🕵️ SPECIALIZED SUB-AGENTS

Invoke explicitly when the task matches the domain. Never default to a generalist approach when a specialist exists.

Core Analysis Agents

Agent	When to Invoke
`codebase_investigator`	Architecture mapping, dependency tracing, root-cause analysis for complex bugs
`cli_help`	Questions about Gemini CLI features, configuration, policies, or behavior
`generalist`	High-volume batch work: mass refactoring, bulk error fixing, research sprints

GSD (Get Shit Done) — Engineering Pipeline

Mandatory pipeline for all professional engineering tasks. Never skip stages.

```
gsd-roadmapper → gsd-planner → [execution loop] → gsd-code-reviewer → gsd-code-fixer → gsd-verifier
```

Agent	Role
`gsd-roadmapper`	Break down goal into a dependency-ordered project plan
`gsd-planner`	Convert roadmap into executable, step-by-step implementation plan
`gsd-code-reviewer`	Structural review — security holes, logic bugs, anti-patterns, unnecessary complexity
`gsd-code-fixer`	Apply reviewer findings atomically — one issue at a time
`gsd-debug-session-manager`	Manage a scientific debugging loop with checkpoints
`gsd-debugger`	Execute individual debug cycles: hypothesis → test → result
`gsd-eval-planner`	Design evaluation strategies for AI features
`gsd-eval-auditor`	Verify that evaluations are sound, not just passing
`gsd-ui-researcher`	Research and define design contracts before UI implementation
`gsd-ui-checker`	Check implementation against design contracts
`gsd-ui-auditor`	6-pillar visual audit: layout, color, typography, motion, accessibility, responsiveness
`gsd-verifier`	**Final gatekeeper** — confirms the goal was achieved, not just that tasks were checked off

> **Rule**: `gsd-verifier` must run at the end of every GSD pipeline. A checklist of completed tasks is NOT a verified outcome.

🛠️ SKILL ACTIVATION GUIDE

Activate the relevant skill **at the start** of any task in its domain — not after starting.

Engineering & Infrastructure

Skill	Activate When
`tdd-workflow`	Any feature or bug fix requiring code changes — enforce ≥ 80% coverage, write tests first
`security-review`	Pre-deploy, auth flows, input handling, secret management, public-facing endpoints
`backend-patterns`	Node.js / Express / API design — follow industry-standard patterns
`frontend-patterns`	Next.js / React component architecture and state management
`mcp-server-patterns`	Building or extending any Model Context Protocol server
`bun-runtime`	Any project using Bun as the runtime or test runner
`devops-pipeline`	CI/CD configuration, Docker, Kubernetes, IaC, deployment automation
`everything-claude-code-conventions`	Commit messages, module organization, naming conventions across the entire codebase
`adr-writer`	Any architectural decision with long-term impact
`i18n-l10n`	Any UI that may scale to non-English markets or multiple locales

Product & Design

Skill	Activate When
`product-capability`	Translating a PRD, brief, or vague feature request into an implementation-ready SRS
`frontend-design`	Any UI component, page, dashboard, or visual interface — bold, distinctive aesthetics over generic AI defaults
`frontend-slides`	HTML presentations, pitch decks, or animation-rich slide content
`design-system`	Creating or updating design tokens, component library, or Storybook documentation

Frontend — Extended Skills

Activate **in addition to**, not instead of, core design/pattern skills.

Skill	Activate When
`frontend-performance`	Any page going to production — Lighthouse score, Core Web Vitals, bundle size
`frontend-seo`	Any public-facing page, landing page, or marketing site indexed by Google
`frontend-animation`	Landing pages, hero sections, scroll reveals, or any motion-driven UI
`frontend-a11y`	Any UI work — WCAG 2.1 AA is mandatory, not optional
`frontend-i18n`	Any user-facing string — if the product may go global, externalize from day 1

Research & Content

Skill	Activate When
`deep-research`	Multi-source research synthesis requiring web tools
`market-research`	Competitor analysis, market sizing, sector research
`investor-materials`	Financial modeling, pitch decks, investor-facing documents
`content-engine`	Drafting platform-native content (articles, posts, threads)
`crosspost`	Distributing content across X, LinkedIn, and other platforms in their native voice
`brand-voice`	Building or applying a writing style profile from real-world examples

---

🔌 EXTENSIONS & MCP SERVERS

Extension: `ralph` — Autonomous Iteration Engine (ALWAYS ACTIVE)

Source: `https://github.com/gemini-cli-extensions/ralph`

Ralph is a self-referential development loop. It intercepts the agent's exit via an `AfterAgent` hook, clears conversational memory (while preserving file state), and restarts the agent with the original prompt until the `--completion-promise` is met or `--max-iterations` is exhausted.

Required configuration in `~/.gemini/gemini.json`:

```
{
  "hooksConfig": {
    "enabled": true
  },
  "context": {
    "includeDirectories": ["~/.gemini/extensions/ralph"]
  }
}
```

> ⚠️ If `includeDirectories` is missing, Gemini CLI will error on startup. Verify this config before every session.

Command / Flag	Purpose
`/ralph:loop "<task>"`	Start an autonomous loop for the given task
`--completion-promise "<S>"`	Loop exits cleanly when agent outputs this phrase (honest signal only)
`--escalation-signal "<S>"`	Loop exits cleanly, preserving memory, when agent outputs this phrase
`--max-iterations <N>`	Hard iteration cap — always set this; default 5, recommend 10–20 for real tasks
`/ralph:cancel`	Safely cancel a running loop without ghost-hijacking your next prompt

Standard Ralph invocation (always include both signals):

```
/ralph:loop "<task description with explicit success criteria>" \
  --completion-promise "TASK_DONE" \
  --escalation-signal "ESCALATE_TO_HUMAN" \
  --max-iterations 10
```

Loop termination rules — three valid exits:

Exit condition	Ralph behaviour	Memory wiped?
Agent outputs `--completion-promise`	Clean exit — task complete	Yes (normal)
Agent outputs `ESCALATE_TO_HUMAN`	Clean exit — paused for human input	**No** ← critical
`--max-iterations` reached	Forced exit — blocked report surfaced to human	**No** ← critical

> ⚠️ **Memory wipe only happens on a loop restart.** Both escalation and max-iterations exits must preserve memory so the human can read the full context of why the agent stopped.

Ghost protection: Ralph detects prompt mismatch on interrupted loops and cleans up silently — it will not hijack the new session.

⚠️ REQUIRED: Patch Ralph's AfterAgent Hook

The `--escalation-signal` flag requires a one-time patch to Ralph's source. Without this patch, Ralph will wipe memory and restart the loop when the agent tries to escalate — swallowing the escalation message entirely.

File to edit: `~/.gemini/extensions/ralph/hooks/after-agent.js`

Replace the existing termination logic with:

```
const COMPLETION_PROMISE = process.env.RALPH_COMPLETION_PROMISE;
const ESCALATION_SIGNAL =
  process.env.RALPH_ESCALATION_SIGNAL || "ESCALATE_TO_HUMAN";

// Read the agent's last output
const output = context.lastAgentOutput ?? "";

if (output.includes(COMPLETION_PROMISE)) {
  // Task genuinely complete — normal exit, memory wipe is fine
  exit({ reason: "completed", preserveMemory: false });
} else if (output.includes(ESCALATION_SIGNAL)) {
  // Agent hit a human escalation trigger — clean pause, never wipe memory
  exit({ reason: "escalated", preserveMemory: true });
} else if (context.iterationCount >= context.maxIterations) {
  // Hard cap reached — surface blocked report, never wipe memory
  exit({ reason: "max_iterations_reached", preserveMemory: true });
} else {
  // Normal loop — restart with memory cleared (intended behaviour)
  restartLoop({ clearMemory: true });
}
```

> **Verify the patch is live** by running a test loop, deliberately outputting `ESCALATE_TO_HUMAN`, and confirming Ralph exits without restarting and your message is visible.

Extension: `gemini-cli-security` (v0.5.0)

Always active for any task involving dependencies, auth, or deployment.

Tool / Skill	Purpose
`securityServer` (MCP)	Core security analysis — invoke before any deploy or PR
`osvScanner` (MCP)	Scan all dependencies against Google OSV vulnerability database
`security-patcher`	Automated remediation of flagged vulnerabilities
`poc`	Generate Proof-of-Concept environments to safely test vulnerabilities
`dependency-manager`	Resolve dependency conflicts in isolated sandboxes — never pollute the main env

> **Rule**: Run `osvScanner` on every new dependency added. Run `securityServer` before every production deployment. Zero HIGH or CRITICAL CVEs before go-live.

Extension: `Stitch` (v0.1.4)

Activate for any UI design or frontend code generation task.

Capability	Use Case
Screen generation from text	Generate full UI screens from a plain-language description
Variant generation	Create multiple design variants for A/B testing or review
Design system management	Maintain tokens, components, and style consistency across the project

> **Rule**: Before writing any UI component manually, check if Stitch can generate a high-quality base. Use `gsd-ui-researcher` → Stitch → `gsd-ui-auditor`.

🔄 MASTER WORKFLOW

Every non-trivial task follows this pipeline. No skipping phases.

```
0. THINK — Before anything else
   ├── State assumptions explicitly — surface tradeoffs
   ├── Identify if a simpler approach exists — push back if so
   ├── Check mandate priority order if conflicts exist
   ├── Check human escalation triggers — if any apply, output ESCALATE_TO_HUMAN before starting Ral
   └── Define success criteria: what does "done" actually look like?
       ├── Define the --completion-promise phrase
       └── --escalation-signal is always "ESCALATE_TO_HUMAN" (non-negotiable)

1. DESIGN (for features touching 3+ components or DB schema)
   ├── Write/update ADR if architectural decision is involved
   ├── Produce sequence diagram, ERD, and API contract
   └── Get confirmation before implementation starts

2. RESEARCH
   ├── Map the codebase (codebase_investigator if large or complex)
   ├── Reproduce the issue or understand the requirement empirically
   └── Identify all relevant files, dependencies, and constraints

3. STRATEGY
   ├── Activate relevant Skills for the domain
   ├── Invoke gsd-roadmapper → gsd-planner
   ├── Define verifiable success criteria per step
   └── Enter plan mode — do not act before the plan is confirmed

4. RALPH LOOP (wraps the entire execution phase)
   └── /ralph:loop "<full task + success criteria>"
         --completion-promise "<PHRASE>"
         --escalation-signal "ESCALATE_TO_HUMAN"
         --max-iterations <N>
       ├── PLAN: Specific implementation and test approach for this step
       ├── ACT: Make surgical, idiomatic changes — one concern at a time
       └── VALIDATE: Run tests, linting, type checks — fix failures at root cause
           ├── Goal met → output completion-promise truthfully → loop exits, memory wiped
           ├── Escalation trigger hit → output ESCALATE_TO_HUMAN + structured message → loop exits,
           └── Max-iterations hit → Ralph exits preserving memory → surface blocked report → wait f

5. REVIEW & CLOSE
   ├── gsd-code-reviewer → gsd-code-fixer (for any non-trivial code)
   ├── visual-regression skill (for any UI changes — run Chromatic/Percy CLI)
   ├── load-testing skill (for any endpoint going to production under load — run k6)
```

```
    ├── gsd-verifier: Confirm the GOAL is achieved, not just the tasks
    └── Security: osvScanner + securityServer before any deployment
```

> **Golden Rule**: Validation is the **only** path to finality. "I think it works" is not done. Tests green + goal confirmed = done.

📐 QUALITY STANDARDS

Code — Universal

- All new code must have tests. Target: **≥ 80% coverage** (`tdd-workflow`). Critical paths: **≥ 95%**.
- Zero suppressed warnings or linting errors in committed code.
- Every function has a **single, clear responsibility**.
- No hardcoded values — use constants, env vars, or config files.
- No abstractions without at least two active consumers.
- If you write 200 lines and it could be 50 — rewrite it before submitting.
- Technical debt must be tracked — open a ticket, add a `// TODO(ticket-id):` comment, never silently leave debt.

UI / Frontend

Run the **6-pillar audit** (`gsd-ui-auditor`) before any UI ships:

1. Layout & Spacing
2. Color & Contrast (WCAG 2.1 AA — contrast ratio ≥ 4.5:1 for normal text, ≥ 3:1 for large text)
3. Typography
4. Motion & Interaction (respect `prefers-reduced-motion`)
5. Accessibility (keyboard nav, ARIA, screen reader — `frontend-a11y` skill mandatory)
6. Responsiveness (mobile-first, tested at 320px, 768px, 1280px, 1440px)

Cross-Browser / Device Matrix (must pass before go-live):

Browser	Versions
Chrome	Latest 2
Firefox	Latest 2
Safari	Latest 2
Edge	Latest 2
Mobile Safari	iOS 15+
Chrome Android	Latest

Lighthouse Targets (production mode — not dev):

- Performance: ≥ 90
- Accessibility: **≥ 95**
- Best Practices: ≥ 95
- SEO: ≥ 90

Core Web Vitals (measured in production — not Lighthouse simulation):

- LCP < 2.5s | CLS < 0.1 | INP < 200ms

Error Handling:

- Every async boundary wrapped in an Error Boundary component with a fallback UI.
- Fallback UI must be user-friendly — never show stack traces or raw error objects to users.
- Log errors to your observability platform (Sentry, Datadog) with context.

i18n / L10n:

- Never hardcode user-facing strings. Use an i18n library (e.g., `next-intl`, `react-i18next`) from day one.
- Date, time, number, and currency formatting must use locale-aware APIs.
- RTL layout support required if the product targets Arabic, Hebrew, Farsi, or Urdu markets.

Bundle:

- Dynamic `import()` for all routes and heavy components.
- Never import an entire library for one function.
- Flag any chunk > 200KB in bundle analyzer.
- All images in WebP/AVIF; always `width` + `height` set; `loading="lazy"` below the fold.

Animation:

- Micro-interactions: 100–200ms | Entrances: 300–500ms | Page transitions: 400–600ms.
- Hard ceiling: 800ms (exception: intentional loading screens).
- Only animate `transform` and `opacity` (GPU-accelerated, no reflow).
- Respect `prefers-reduced-motion` — always.
- Replay all animations at 0.25x speed in Chrome DevTools. Anything cheap at slow speed is cheap at full speed — fix before shipping.

Design System

- **Single source of truth**: Design tokens defined in Figma and mirrored exactly in code (`tokens.css` or `tokens.ts`). Any drift is a bug.
- **Component library**: Every shared component documented in Storybook with props, variants, and accessibility notes.
- **Design QA**: Designer signs off on implementation before merge. Use `gsd-ui-checker` to generate a diff report.
- **Versioning**: Design system has its own version number. Breaking changes require a major bump and a migration guide.

Backend & API

- Every public endpoint: input validation + server-side sanitization (Zod, Joi, or equivalent).
- Authentication checked at the middleware layer — never inside individual handlers.
- No N+1 queries — use query optimization, eager loading, or batching.
- All DB migrations are reversible. Test rollback before deploying.
- API contracts documented (OpenAPI/Swagger) — breaking changes are versioned.
- **Rate limiting** on all public-facing endpoints (e.g., express-rate-limit, Redis-based for distributed systems).
- Pagination on all list endpoints — no unbounded queries. Maximum page size enforced server-side.
- **DB indexing review**: Every query in a new feature reviewed for missing indexes. EXPLAIN ANALYZE run on slow queries.

Caching Strategy:

Layer	Tool	Use Case
CDN	Cloudflare / CloudFront	Static assets, public API responses with long TTL
Application	Redis	Session data, rate limit counters, expensive computed results
In-memory	Node.js LRU cache	Hot lookup tables, config values fetched repeatedly
DB query cache	Prisma / ORM layer	Repeated identical queries within a request lifecycle

- Cache invalidation must be explicit — never rely on TTL alone for critical data.
- Never cache responses that include user-specific sensitive data without scoping by user ID.

Async Jobs / Queues:

- Use a proper job queue (BullMQ, Sidekiq, SQS) for any work that takes > 200ms or can fail and retry.
- Jobs must be idempotent — safe to run twice.
- Dead letter queues required for all production job queues.
- Job failures must alert — never silently fail.
- Long-running jobs must expose a status endpoint or WebSocket update.

API Contract Testing:

- Use Pact or similar for consumer-driven contract tests between services.
- Contract tests run in CI before integration tests.
- Breaking contract changes block the pipeline.

DevOps & Infrastructure

- No secrets in code, environment files committed to version control, or logs.
- Infrastructure defined as code (Terraform / Pulumi / CDK) — no manual console changes.
- Every deployment is reproducible from version control alone.
- Blue/green or rolling deployments — zero-downtime mandatory for production.
- Rollback procedure documented, tested, and executable in < 5 minutes.
- All services containerized (Docker) — no "works on my machine" deployments.

Environment Promotion Strategy:

```
feature branch → dev (auto-deploy on merge) → staging (manual promote) → prod (manual promote + app
```

- `dev` : mirrors `main` . Runs full test suite. No approval needed.
- `staging` : production-identical infrastructure. Smoke tests run automatically post-deploy.
- `prod` : requires 2-person approval. Post-deploy smoke tests mandatory. Rollback plan confirmed before promoting.

CI Pipeline (all gates must pass before merge):

```
lint → type-check → unit tests → integration tests → contract tests → build → security scan (osvSca
```

- Branch protection: `main` requires PR + passing CI + at least 1 approval.

- No force-pushes to `main` or `staging`. Ever.
- Performance budget check: bundle size, Lighthouse score — regression blocks merge.

Secrets Management:

- Dev: `.env.local` (never committed). Template: `.env.example` (committed, no real values).
- Staging/Prod: secrets injected at runtime via Vault, AWS Secrets Manager, or equivalent.
- Secret rotation policy: credentials rotated every 90 days minimum, immediately on suspected exposure.
- Automated dependency PRs via Dependabot or Renovate — reviewed weekly, merged within 2 weeks for non-breaking updates.

Docker:

- Multi-stage builds — production image contains only runtime artifacts.
- Non-root user in all containers.
- `.dockerignore` excludes `.env`, `node_modules`, `.git`, build artifacts.
- Pin base image versions — never use `:latest` in production Dockerfiles.
- Secrets injected at runtime via env — never baked into image layers.

Kubernetes (if applicable):

- Resource requests AND limits defined for every container.
- Liveness and readiness probes on every service.
- Horizontal Pod Autoscaler configured for stateless services.
- Network policies enforced — default deny, explicit allow.
- Secrets stored in Kubernetes Secrets or external vault (not ConfigMaps).

Observability (non-negotiable before go-live):

- Structured logs (JSON) with correlation IDs across all services.
- Metrics: error rate, latency (p50/p95/p99), throughput, saturation.
- Alerts defined for: error rate > 1%, p99 latency > SLO, pod restarts, disk > 80%, job queue depth spike.
- Distributed tracing for any multi-service request path (OpenTelemetry).

SLO / SLA Definitions (define before go-live):

- Availability SLO: e.g., 99.9% uptime (< 8.7 hours downtime/year)
- Latency SLO: e.g., p99 < 500ms for API endpoints
- Error rate SLO: e.g., < 0.1% 5xx errors over rolling 7-day window
- SLO breach triggers an incident — on-call is paged.

Runbooks:

- Every critical service has a runbook in `/docs/runbooks/`.
- Runbook covers: symptoms, triage steps, common fixes, escalation contacts.

- Runbooks tested in staging during game days — not for the first time in a real incident.

Incident Response (execute directly — no extension required):

- Severity levels: P0 = production down, P1 = major feature broken, P2 = degraded, P3 = minor.
- On-call rotation documented and tested before go-live.
- On incident: open `/docs/runbooks/<service>.md` → follow triage steps in order → don't improvise.
- Declare incident in your comms channel immediately (don't wait to be sure — declare early, downgrade later).
- Mitigation first, root cause second. Rollback is always a valid first move.
- P0/P1: postmortem required within 72 hours — blameless, structured, action-item focused.
- Postmortem template at `/docs/runbooks/postmortem-template.md`.

Load Testing (execute directly using k6 — no extension required):

- Activate before any production release for endpoints expected to handle > 100 req/s.
- Install: `brew install k6` or `apt install k6` or `docker run grafana/k6`.
- Write test script targeting the endpoint (save to `/tests/load/<endpoint-name>.js`):

```
import http from "k6/http";
import { check, sleep } from "k6";

export const options = {
  vus: 200,
  duration: "5m",
  thresholds: {
    http_req_failed: ["rate<0.001"], // error rate < 0.1%
    http_req_duration: ["p(99)<500"], // p99 < your SLO
  },
};

export default function () {
  const res = http.get("https://your-endpoint.com/api/v1/resource");
  check(res, { "status 200": (r) => r.status === 200 });
  sleep(1);
}
```

- Run: `k6 run --out json=docs/load-tests/$(date +%F)-results.json tests/load/<endpoint-name>.js`
- Pass criteria: all thresholds green, no memory leak trend in container metrics.
- Compare results against previous run — any regression in p99 or error rate blocks deploy.

Security

- No secrets in code, logs, or version control — **ever**.
- All user inputs sanitized and validated **server-side** (client-side is UX only).
- Auth flows reviewed against `security-review` checklist before every release.
- OWASP Top 10 reviewed for every public-facing feature.
- Dependencies scanned with `osvScanner` on every addition.
- Principle of least privilege: every service/user/role has only what it needs.
- SQL queries use parameterized statements or ORM — never string interpolation.
- HTTPS enforced everywhere. HSTS headers set. No mixed content.

Required HTTP Security Headers (every response):

```
Content-Security-Policy: default-src 'self'; script-src 'self' [approved CDNs]
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
```

CORS Policy:

- Allowlist only known origins — never use wildcard `*` for credentialed requests.
- Pre-flight caching: `Access-Control-Max-Age: 86400`.
- CORS config reviewed as part of `security-review` checklist.

Data Privacy (GDPR / applicable regulations):

- PII must be identified and documented in a data map.
- Data retention policy defined per data type — delete or anonymize at expiry.
- Right to erasure: user data deletion pipeline must exist before launch.
- No PII in logs, analytics events, or error reports.
- Cookie consent implemented if using non-essential cookies or tracking.
- DPA (Data Processing Agreement) required with all third-party processors.

Pre-Deploy Security Gate:

- `osvScanner` → zero HIGH or CRITICAL CVEs
- `securityServer` → all findings resolved or risk-accepted with written documentation
- Auth flow tested: login, logout, session expiry, token refresh, privilege escalation attempt
- All environment variables audited — none committed to version control
- Penetration testing required before first public launch and annually thereafter

SEO (Full Standard — Public Pages)

Required on every public-facing page:

- Unique `<title>` (50–60 chars) — must include the primary keyword.
- Unique `<meta description>` (120–160 chars) — compelling, not keyword-stuffed.
- Canonical tag: `<link rel="canonical" href="...">`.
- `robots: index, follow` (or explicitly `noindex` where appropriate).
- One and only one `<h1>` per page — must contain the primary keyword.
- Logical heading hierarchy: `h1 → h2 → h3`, never skip levels.

Open Graph / Social:

- `og:title`, `og:description`, `og:image` (1200×630px min, < 8MB), `og:url`, `og:type`.
- `twitter:card`, `twitter:title`, `twitter:description`, `twitter:image`.
- Test OG tags with Meta Sharing Debugger before launch.

Structured Data:

- JSON-LD in `<head>` for Organization, WebPage, Article, Product, or BreadcrumbList as applicable.
- Validate with Google Rich Results Test — zero errors before go-live.

Technical SEO:

- `sitemap.xml` generated automatically on build, submitted to Google Search Console.
- `robots.txt` at root — reviewed before go-live.
- 404 page returns HTTP 404 status (not 200). Redirects use 301, not 302, for permanent moves.
- No orphan pages — every page reachable via internal links.
- Core Web Vitals must be in the "Good" threshold in CrUX (real user data), not just lab scores.

Validation Gate: Zero critical errors in Google Search Console before go-live.

Commits

- Follow `everything-claude-code-conventions` for commit message format.
- Each commit is **atomic**: one logical change, fully tested, fully working.
- Never commit broken code to main/master.
- PR description includes: what changed, why, how to test, any migration steps, and rollback plan.

🚀 Skill Details — Frontend Performance

Activate: Any page going to production or when Lighthouse / Core Web Vitals matter.

Rendering Strategy (Next.js):

- Static pages (marketing, landing): SSG via `getStaticProps`
- Dynamic but cacheable: ISR via `revalidate`
- Truly dynamic (dashboards, auth): SSR or client-side with skeleton loaders
- Never use SSR for pages that could be static

Caching & Headers:

- `Cache-Control: public, max-age=31536000, immutable` for all hashed static assets
- `Cache-Control: no-store` for API responses with sensitive data
- Use a CDN for all static assets in production

Fonts:

- `font-display: swap` on all custom fonts
- Preload critical fonts with `<link rel="preload">`
- Subset fonts to only characters used

🎬 Skill Details — Frontend Animation

Library Selection:

Use Case	Library
Page transitions, component motion	Framer Motion (default)
Complex timeline / SVG animations	GSAP
Scroll-triggered animations	GSAP ScrollTrigger
Simple hover/fade	Tailwind transitions
Pre-made designer animations	lottie-react

Standard Framer Motion Patterns:

```
const fadeUp = {
  hidden: { opacity: 0, y: 24 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.5, ease: "easeOut" } },
};

const container = {
  hidden: {},
  visible: { transition: { staggerChildren: 0.1 } },
};

<motion.div
  initial="hidden"
  whileInView="visible"
  viewport={{ once: true, margin: "-80px" }}
  variants={fadeUp}
/>;
```

👁️ Skill Details — Visual Regression Testing

Activate: Any UI change before merge. Run this — don't eyeball it.

Using Chromatic (Storybook-based — preferred for component libraries):

```
# Install once
npm install --save-dev chromatic

# Run on every UI PR
npx chromatic --project-token=<your-token> --exit-zero-on-changes
```

- Chromatic captures a snapshot of every Storybook story and diffs against baseline.
- Review diffs in the Chromatic web UI — approve intentional changes, reject regressions.
- Pass criteria: zero unreviewed visual changes before merge.
- Set `--auto-accept-changes` only on the `main` branch to update baselines after approval.

Using Percy (app-level — preferred for full page regression):

```
# Install once
npm install --save-dev @percy/cli @percy/playwright

# Run with your e2e suite
npx percy exec -- playwright test
```

- Percy diffs full pages against baseline at multiple viewport widths.
- Pass criteria: zero unreviewed diffs before merge.

Minimum viewport widths to capture: 375px (mobile), 768px (tablet), 1280px (desktop).

Rule: Visual regression runs in CI on every PR touching UI files. A diff that isn't reviewed is a regression waiting to ship.

🛡️ Skill Details — Security Review

Activate: Pre-deploy, auth flows, input handling, public-facing endpoints.

OWASP Top 10 Checklist:

- [ ] Broken Access Control — verify every route enforces authorization
- [ ] Cryptographic Failures — no sensitive data in plaintext; TLS everywhere
- [ ] Injection — parameterized queries only; sanitize all inputs
- [ ] Insecure Design — threat modeled before implementation
- [ ] Security Misconfiguration — default credentials changed; debug mode off in prod
- [ ] Vulnerable Components — `osvScanner` clean
- [ ] Auth Failures — session tokens invalidated on logout; MFA available
- [ ] Software Integrity Failures — dependency checksums verified; no unpinned deps
- [ ] Logging Failures — sufficient logs for forensics; no sensitive data in logs
- [ ] SSRF — outbound requests validated and allowlisted

⚙️ Skill Details — DevOps Pipeline

Activate: CI/CD, Docker, Kubernetes, IaC, deployment automation.

CI Pipeline (mandatory gates — all must pass before merge):

```
lint → type-check → unit tests → integration tests → contract tests → build → security scan (osvSca
```

🧠 AGENT SELECTION CHEATSHEET

```
Got a vague bug?              → codebase_investigator
Building a feature?           → ADR (if architectural) → gsd-roadmapper → gsd-planner → tdd-workflo
Reviewing code?               → gsd-code-reviewer → gsd-code-fixer
Debugging a hard issue?       → gsd-debug-session-manager → gsd-debugger → /ralph:loop

Building UI?                  → gsd-ui-researcher → frontend-design → Stitch → gsd-ui-auditor → vis
UI has motion?                → frontend-animation (Framer Motion / GSAP)
UI is public-facing?          → frontend-a11y (WCAG 2.1 AA) + frontend-seo + frontend-i18n
Going to production?          → frontend-performance (Lighthouse ≥ 90) → load-testing (skill)
Full landing page?            → frontend-design + frontend-animation + frontend-performance + front

API design?                   → ADR → backend-patterns → tdd-workflow → security-review → /ralph:lo
Database changes?             → backend-patterns (verify migration is reversible + indexes reviewed
Adding a dependency?          → osvScanner first, always
Caching needed?               → backend-patterns (see caching strategy table)
Heavy async work?             → backend-patterns (job queue: BullMQ/SQS)

Deploying?                    → security-review → osvScanner → securityServer → load-testing (skill
Active incident?              → incident-response (skill) → runbook → postmortem
Writing content?              → brand-voice → content-engine → crosspost
Researching a topic?          → deep-research
Building a pitch/deck?        → market-research → investor-materials → frontend-slides

Architectural decision?       → ADR first, always
Anything non-trivial?         → THINK → check escalation triggers → plan → /ralph:loop
Done with a task?             → gsd-verifier (always last)
Mandates in conflict?         → check priority order → escalate if unresolved
```

ANTI-PATTERNS — NEVER DO THESE

Anti-Pattern	Why It's Wrong
Silently picking one interpretation	You may build the wrong thing entirely
Writing code before defining success	You won't know when to stop or whether it worked
"Improving" untouched code in a diff	Creates noise, risk, and reviewer distrust
Deleting pre-existing dead code unasked	It may be referenced elsewhere or kept for a reason
Altering a failing test to make it pass	Hides bugs; ships broken behavior with false confidence
String interpolation in SQL queries	SQL injection — inexcusable
Secrets in code or committed `.env`	Security catastrophe
Animating `width`, `height`, or `margin`	Layout reflow, jank, poor Lighthouse score
Deploying without running `osvScanner`	Shipping known vulnerabilities
Deploying without a rollback plan	One bad deploy can take down production with no recovery path
Unbounded DB queries without pagination	Will work in dev, will destroy prod under load
Using `:latest` Docker tags in production	Breaks reproducibility; unknown regressions on re-deploy
"I think it works" as a sign-off	Not validation. Run the tests. Confirm the goal.
Running a task without `/ralph:loop`	One-shot turns miss errors, skip retries, and skip self-correction
Outputting completion-promise to escape	Dishonest loop exit; ships broken behavior with false confidence
Outputting ESCALATE_TO_HUMAN + completion-promise together	Mutually exclusive signals — one means done, one means paused. Never combine.
Escalating without outputting ESCALATE_TO_HUMAN phrase	Ralph won't recognize it as a clean exit — it will restart and swallow your message
Patching Ralph's AfterAgent hook incorrectly	Escalation silently fails; the AI loops forever instead of surfacing the human decision needed

Anti-Pattern	Why It's Wrong
Skipping ADR for architectural decisions	Future engineers have no context for why decisions were made
Hardcoding user-facing strings	Impossible to localize later without a full rewrite
Caching without explicit invalidation	Stale data bugs that are nearly impossible to reproduce
No error boundary on async UI	One failed fetch crashes the entire page for the user
Skipping load testing before launch	Discovering your capacity ceiling in production, under real users
PII in logs or analytics events	GDPR violation; legal exposure; user trust destroyed
Wildcard CORS for credentialed requests	Any malicious site can make authenticated requests on behalf of your users
No runbook before going on-call	First time you read the runbook is during a P0 at 3am
Skipping visual regression on UI changes	Shipping broken layouts you didn't notice because you only tested your own screen. Run Chromatic/Percy.
Making irreversible changes autonomously	Always escalate — get explicit human confirmation first

---

Last updated: 8 May 2026 — v2.0.0. Full production-grade rewrite. Added: ADRs, mandate priority, human escalation protocol, Ralph failure protocol, WCAG 2.1 AA, cross-browser matrix, error boundaries, i18n/l10n, design system standards, caching strategy, async job standards, API contract testing, DB indexing review, environment promotion strategy, secrets management, Dependabot policy, load testing (load-tester agent), visual regression (visual-regression agent), incident response (incident-responder agent), SLO/SLA definitions, runbook requirements, full SEO standard, GDPR/data privacy, CORS policy, security headers, tech debt tracking, performance regression in CI, and changelog.